

Ferrari Challenge: TIME ATTACK

Technical Report and User Manual

Alessio Pannozzo

Matricola: 1960374

Contents

1	Introduction	2
2	Environment and Third-Party Resources	2
2.1	Development Environment	2
2.2	Libraries and Tools	2
2.3	External Assets	2
3	Technical Architecture and Implementation	3
3.1	Game Loop and State Management	3
3.2	Circuit Optimization and Physics Generation	3
3.3	Vehicle Dynamics	3
3.4	Character and Procedural Animation	4
3.5	Lighting, Particles, and Audio	4
3.6	Dynamic Camera System and Customization	4
4	Interactions and Game Mechanics	5
4.1	Time Attack Logic and Pit Lane	5
4.2	Fatal Crash System	5
5	User Manual	5
5.1	Spawn Modes	5
5.2	Controls	5
5.2.1	Racer Controls (On Foot)	5
5.2.2	Vehicle Controls (Driving)	5
5.2.3	System Controls	6

1 Introduction

The *Ferrari Challenge Time Attack* is a 3D web-based racing simulator. The primary objective of the game is to pilot a Ferrari LaFerrari around the Red Bull Ring circuit in Spielberg, Austria, aiming for the fastest lap time. The project bridges high-fidelity graphics with real-time rigid-body physics, offering an immersive experience that includes both driving and on-foot exploration.

2 Environment and Third-Party Resources

2.1 Development Environment

The project is built entirely as a client-side web application utilizing WebGL for rendering. The architecture relies on sequential script loading and ES modules to orchestrate the game logic within the browser.

2.2 Libraries and Tools

The core logic of the game was built using the following libraries:

- **Three.js (r128):** Used as the primary 3D rendering engine. It handles the WebGL pipeline, scene graph management, lighting, and shadow mapping.
- **Cannon-es (0.20.0):** A lightweight 3D physics engine for the web. It calculates real-time rigid-body dynamics, vehicle suspension via raycasting, and collision detection.
- **Tween.js (18.6.4):** Utilized for procedural animations, such as the skeletal animation of the character and cinematic camera movements.

2.3 External Assets

All 3D models and audio files were sourced externally but underwent heavy optimization to function within a real-time web environment. The complete list of resources and their original sources is detailed below:

Asset	Source Link
3D Models	
Car model	View on Fab.com
Red Bull Ring (Spielberg)	View on Fab.com
Racer character	View on Fab.com
Audio	
Engine idle sound	View on Freesound.org
Tire screech sound	View on Freesound.org

Pipeline Tool: Blender was utilized extensively to process the 3D assets, performing custom rigging, polygon decimation, and texture downscaling prior to engine integration.

3 Technical Architecture and Implementation

3.1 Game Loop and State Management

The application follows a strict state machine architecture transitioning through five phases: *loading*, *ready*, *walking*, *playing*, and *gameover*. The core orchestrator ('Game' class) manages a 'requestAnimationFrame' loop that updates both the logic and rendering. To ensure deterministic physics behavior, the Cannon-es world is stepped at a fixed rate of 1/60th of a second, decoupling the physics calculations from the visual frame rate.

3.2 Circuit Optimization and Physics Generation

The original track model weighed approximately 300MB and featured 8K (8192) resolution textures, which was unsuitable for web delivery. Using a custom Blender pipeline, the triangle count was reduced, surfaces were normalized, and textures were downscaled, resulting in a 100MB footprint.

To optimize the physics engine, the track geometry is algorithmically filtered by the 'EnvironmentManager'. Meshes are categorized by their naming conventions:

- **ROAD/OFFROAD:** Converted to static 'CANNON.Trimesh' bodies to act as drivable surfaces.
- **WALL:** Converted to solid collision barriers.
- **SKIP:** Decorative elements (e.g., trees, lights, crowds) are ignored by the physics engine entirely to save CPU cycles.

3.3 Vehicle Dynamics

The vehicle is simulated using Cannon's 'RaycastVehicle'. The hierarchical model separates the main chassis from the four independent wheels.

- **Chassis and Armor:** The main chassis is simulated as a 1500 kg dynamic body. Initially, bounding the car with a standard `CANNON.Box` might seem logical; however, the Cannon-es engine suffers from known inaccuracies when calculating continuous collision detection between a `Box` and complex `Trimesh` geometries (which are used for the track walls) at high speeds. This limitation often causes the box geometry to snag on the edges of wall polygons or glitch completely through the barriers. To circumvent this physics engine bottleneck, the vehicle's chassis was wrapped with four invisible `CANNON.Sphere` colliders positioned at the corners. Spheres utilize a much simpler and mathematically robust radius-based calculation for collision intersections. This engineering choice ensures that the vehicle slides smoothly along the track barriers and completely prevents high-speed tunneling during severe impacts.
- **Raycast Override:** A critical optimization was implemented by overriding the vehicle's 'castRay' method. Custom logic forces the suspension rays to ignore complex wall geometry and checkpoints, interacting exclusively with the road surface.

- **Handling:** Acceleration is applied progressively up to a maximum force of 6000 N, and braking is distributed at a 60/40 ratio between the front and rear wheels. Steering responsiveness dynamically decreases as vehicle speed increases to maintain stability. Simulated aerodynamic downforce is also applied proportionally to the vehicle's speed.

3.4 Character and Procedural Animation

The racer character was originally a static asset. A full skeletal rig consisting of 21 mapped bones was built from scratch using Blender. Within the engine, the character features zero pre-baked animations. Instead, the 'RacerController' utilizes Tween.js to dynamically calculate sine and cosine waves that drive the bone rotations for walking, running, and idle states procedurally. The physical representation of the character is a lightweight 70 kg 'CANNON.Sphere' representing the feet, allowing smooth traversal over terrain.

3.5 Lighting, Particles, and Audio

The scene is illuminated using a combination of a 'HemisphereLight' for ambient scattering and a 'DirectionalLight' targeting the environment to cast real-time shadows using a 2048x2048 shadow map. Dynamic audio uses 'PositionalAudio' attached to the vehicle mesh; the engine pitch scales dynamically with vehicle speed, and tire screech volume reacts to skid information calculated by the suspension system. Additionally, a programmatic particle system generates smoke sprites when hard braking is detected at high speeds.

3.6 Dynamic Camera System and Customization

The application features a sophisticated camera system tailored for high-speed driving and maximum visual comfort. During the *playing* state, a follow camera continuously calculates an ideal target position relative to the vehicle by utilizing specific distance and height offsets. To ensure visual smoothness, the camera's position is interpolated using a mathematical *lerp* (linear interpolation) function combined with a time-scaled exponential factor. This allows the camera to trail the vehicle smoothly through corners rather than locking rigidly to the chassis.

Furthermore, comprehensive user customization is supported via an interactive camera settings panel. By clicking the dedicated camera button in the pause menu, players can dynamically modify the camera's distance and height parameters using real-time sliders. Crucially, even when the physics engine and game timer are paused, the camera's rendering loop continues to update. This provides the player with immediate visual feedback of the adjustments, allowing them to precisely tailor the field of view and driving perspective to their exact preferences before resuming the session.

4 Interactions and Game Mechanics

4.1 Time Attack Logic and Pit Lane

The track is divided into three sectors using invisible ‘CANNON.Body’ triggers acting as checkpoints. The system records sector times, calculates the final lap time, and tracks the player’s personal best. A geometrically defined pit lane enforces racing rules: if the player exceeds 60 km/h within the pit lane boundaries, a 10-second penalty is applied and a programmatic pit limiter drastically reduces engine force.

4.2 Fatal Crash System

The vehicle chassis includes a collision event listener. If an impact velocity greater than 15.0 m/s (approx. 54 km/h) against a solid object is detected, the game transitions to the ‘gameover’ state, instantly halting the vehicle and terminating the audio playback.

5 User Manual

5.1 Spawn Modes

Upon launching the game, the player can choose between two immersive starting modes:

- **Pit Start:** Spawns the player on foot in the pit lane. This focuses on immersion, allowing the player to walk around the environment and manually enter the car.
- **Grid Start:** Spawns the player directly inside the car on the starting line, with the engine running and ready for immediate racing action.

5.2 Controls

The interaction architecture relies on a lag-free input mapping system.

5.2.1 Racer Controls (On Foot)

- **W / S:** Move forward / Move backward.
- **A / D:** Turn left / Turn right.
- **Shift:** Run (increases movement speed from 4.5 m/s to 8.5 m/s).
- **E:** Enter vehicle (prompt appears when within 3.5 meters of the car).
- **R:** Respawn character to default location.

5.2.2 Vehicle Controls (Driving)

- **W / S:** Accelerate / Reverse (Engine Brake).
- **A / D:** Steer left / right.
- **Space:** Apply strong brakes.

- **E:** Exit vehicle (only available when the vehicle is stationary).
- **R:** Respawn car to the last active spawn point, resetting velocities and penalties.

5.2.3 System Controls

- **V:** Toggle Free Camera mode (allowing the user to detach the camera and fly around the scene using WASD and Mouse look).
- **Esc:** Pause / Resume the game.